Index

A

- A2A
  - *See* Agent-to-Agent Protocol (A2A)
- ACP
  - *See* Agent Communication Protocol (ACP)
- Adapter-based fine-tuning 46
- Agent Communication Protocol (ACP) 560, 653
- Agentic AI system
  - agentic AI—systems 662
  - development lifecycle 663
  - domain adaptation 661
  - fine-tuning 661
  - prompt engineering 661
  - scratch 662
- Agentic systems
  - advisors 324
  - AI chatbots 288
  - autonomous agents 287
  - benefits 321
  - characteristics 287
  - components 289
  - components/interactions 286
  - consistent API across 322
  - evaluator-optimizer pattern
    - commandLineRunner method 318
    - concurrent feedback loop 318
    - dual-layered process 319
    - evaluate() method 318
    - evaluation 319
    - evaluation criteria 315
    - EvaluatorOptimizerWorkflow class 316–318

    - flow process 314–315
    - iterative refinement 315, 319
    - loop() method 318
    - solution and feedback 318
    - source code 315–316
    - tasks benefit 315
- factors 289
- fixed workflows *vs*. dynamic agents 324
- fundamental patterns 285
- guardrails 291
- intelligent agents 287
- key components 285
- knowledge and memory 290
- language model/prompt 288
- latency *vs*. accuracy 323
- lifecycle 319
    - action execution 320
    - initialization 319
    - input reception and perception 320
    - memory update 321
    - observation and feedback 320
    - reasoning and planning 320
    - response generation 321
    - termination or continuation 321
- model portability 321
- models 290
- orchestration tools 291
- Orchestrator-Workers pattern
    - adaptive problem-solving 308
    - ChatClient file 309–310
    - commandLineRunner method 313
    - concurrency 314
    - concurrent worker execution 313
    - process() method 313
    - scalability 314
    - source code 309–312
    - synthesis 314
    - task decomposition 314
    - task decomposition/execution process 307
    - WorkerResponse method 313

    - worker response collection 313
- parallel processing 323
- practices and recommendations 323
- recommendation systems 288
- reliability 323
- robots 288
- routing workflow
    - accurate classification 303
    - categories 303–305
    - determination 305
    - dynamic classification 307
    - execution 305
    - flexible and scalable 307
    - folw chart 302
    - LLM decision 305
    - RoutingWorkflow 305–307
    - RoutingWorkflow class 305
    - source code 303
    - specialized processing 303
    - task routing 307
- start simple 323
- structured output 322
- tools 290
- trade-offs 323
- types of 287
- virtual assistants 288
- workflows and patterns
    - aggregation 302
    - chain workflow pattern 294
    - commandLineRunner method 300, 301
    - concepts 294
    - DemoApplication.java 295–298
    - dynamic systems 293
    - extensibility 298
    - factors 294
    - frameworks 292
    - hybrid systems 292, 293
    - key principles 298
    - parallel execution 301
    - parallelization 299–301

    - parallelStream() method 301
    - responsibility 298
    - routing workflow pattern 302–307
    - sectioning/voting 299
    - sequential dependency 298
    - sequential steps 295
    - stepwise dependency 295
    - trading latency 295
    - workflows 292
- Agent Network Protocol (ANP) 561, 653
- Agent-to-Agent Protocol (A2A) 561, 653
- AGI
    - *See* Artificial general intelligence (AGI)
- AI
    - *See* Artificial intelligence (AI)
- AIDA
    - *See* Artificial Intelligence and Data Act (AIDA)
- Amazon Bedrock service (AWS)
    - certification process/information 465, 466
    - documentation 463
    - features 465
    - foundation models
        - access granted page 472
        - activated playground 474
        - foundation models 467
        - license agreement 467
        - model access page 469
        - model selection 473
        - Nova model 467, 468, 470, 471
        - playground page 472
        - request model access 468
        - response 474
        - security/privacy 462
    - GCP
        - *See* Google Cloud Platform (GCP)
    - guardrails
        - *See* Guardrails
    - home page 464
    - navigation search 463, 464

- RAG
  - *See* Retrieval-augmented generation (RAG)
- SDK project
  - access/secret key 485
  - application.properties file 476, 480
  - approaches 475
  - AwsConfig.java 477
  - BedrockService.java 482
  - build.gradle file 475, 476
  - client.converse method 481
  - controller layer 483
  - converse method 481
  - ConverseRequestDto.java 477
  - ConverseResponseDto.java 478
  - cross-region inference 479, 480
  - message object 481
  - fine-grained access policy 484–485
  - local code 484
  - modelId/inference properties 481
  - model ID 479
  - security credentials 485
  - service component 480
  - service layer 478
  - Spring Initializr dependencies 475
- sign up 463
- Spring AI integration
  - application architecture/control flow 524
  - application.properties file 525
  - ChatModel 527–530
  - inference page 526
  - properties 526
  - Spring Initializr setup 524, 525
- unavailability 463
- ANP
  - *See* Agent Network Protocol (ANP)
- Apache OpenNLP 645–647
- APE
  - *See* Automatic prompt engineering (APE)
- APIs

- *See* Application programming interfaces (APIs)
- Application programming interfaces (APIs) 15
  - Hugging Face 618–621
  - Ollama 71–79
  - OpenAI
    - *See* OpenAI API
  - tool calling 261
- Artificial general intelligence (AGI) 15
- Artificial intelligence (AI)
  - Agentic AI
    - *See* Agentic systems
  - chat playground
    - base model deployment 548
    - code generation section 552
    - deployment details 550
    - details 551
    - selection section 549
  - foundry services 544, 545
  - guardrails
    - *See* Guardrails
  - ML
    - *See* Machine learning (ML)
  - OpenAI service
    - application architecture/control flow 552
    - application.properties 557
    - build.gradle file 553
    - chat playground 547
    - ChatRequestUserMessage 554
    - creation page 545, 546
    - deployment 557
    - GenAiService.java 554, 555
    - subscription/resource group 546
  - prompt engineering 173
  - Spring AI
    - *See* Spring AI ecosystem
- Artificial Intelligence and Data Act (AIDA) 674
- Automatic prompt engineering (APE) 209–213
- AWS
  - *See* Amazon Bedrock service (AWS)

B

- Bedrock service
  - *See* Amazon Bedrock service (AWS)

C

- CFRs
  - *See* Code for Federal Regulations (CFRs)
- Chain of thought (CoT) prompting 200–202
- CI/CD
  - *See* Continuous integration/continuous delivery (CI/CD); Continuous integration/continuous development (CI/CD)
- Code for Federal Regulations (CFRs) 465
- Code prompting 223–233
- Continuous integration/continuous delivery (CI/CD) 667
- Continuous integration/continuous development (CI/CD) 444

D

- Deep Java Library (DJL)
  - architecture 622
  - framework 621
  - hosting/inference 623–625
  - Java developers 670
  - Linnerud dataset 625–627
  - machine learning (ML) 669
  - optimal performance 622
- DJL
  - *See* Deep Java Library (DJL)
- DL4J
  - *See* Eclipse Deeplearning4j (DL4J)

E

- Eclipse Deeplearning4j (DL4J) 670
  - frameworks 643
  - modular libraries 643
  - open-source/community 645

- training/fine-tuning models 644
- use of 645
- Embeddings 131–132

F

- Fine-tuning techniques 45
  - dialogue/instruction tuning 45
  - PEFT techniques 46
  - RLHF model 46
  - safety tuning 46
  - SFT approach 45
- Function calling
  - *See* Tool calling)

G

- GCP
  - *See* Google Cloud Platform (GCP)
- GEMM
  - *See* General Matrix Multiplication (GEMM) operations
- Gen AI
  - *See* Generative AI (Gen AI)
- General Matrix Multiplication (GEMM) operations 393
- Generative AI (Gen AI)
  - adaptation 678
  - agent frameworks 672
  - abstractions 462
  - agent frameworks 672
  - AI
    - *See* Artificial intelligence (AI)
  - Azure AI
    - *See* Artificial intelligence (AI)
  - Bedrock service
    - *See* Amazon Bedrock service
  - capabilities/features 461
  - cloud platforms 461
  - core capabilities 462
  - creations 1

- culture learning 678
- domain specialization 673
- fundamentals 677
- Governance and ethics 675–677
- interoperability protocols 650
- LLMs
  - *See* Large language models (LLMs)
- multimodality 672
- navigation/challenges 657
  - developers 660
  - energy/environmental cost 659
  - ethical/bias mitigation 658
  - factual integrity 659
  - governance/compliance 658
  - jailbreaking 657
  - privacy implications 658
  - prompt injection 657
- paradigms/trends 649
- Quarkus
  - *See* Quarkus application
- regulations/acts
  - builders 674
  - content-labeling 674
  - formal standards 674
  - general-purpose model 673
  - legal environment 673–674
  - systemic risks 673
  - test protocols 674
  - text-and-data-mining (TDM) clauses 674
- Self-hosting LLMs 51
- tool calling
  - *See* Tool calling
- Generative Pre-trained Transformer (GPT) 16
- Google Cloud Platform (GCP)
  - application architecture/control flow 537
  - build.gradle file 535–536
  - configuration arguments 537
  - GenAiController.java 539
  - GenAiRequest.java 539
  - GenAiService.java 538

- service class 537
- Vertex AI platform 530
  - agent builder services 534
  - API enable page 540
  - API key page 541–542
  - base setup 535
  - code feature 533
  - curl requests 543
  - environment variable 542–543
  - model garden 531
  - prompt page 532
  - studio services 532
  - tools section 531
- GPT
  - *See* Generative Pre-trained Transformer (GPT)
- GPU
  - *See* Graphical processing unit (GPU)
- Graphical processing unit (GPU) 4
- Guardrails
  - application.properties file 503–506
  - automated Reasoning 487
  - BedrockService.java 504
  - certification information 499
  - certification technical details 490
  - content filters 490
  - contextual grounding 486
  - ConverseRequest 504
  - critical checkpoints 486
  - denied topics configuration 491
  - different detections 501
  - exclusion 487
  - grounding check configuration 494
  - harmful categories options 489
  - listing overview 503
  - logic-driven verification 487
  - metadata 488
  - multimodal filters 487
  - Nova Micro model 497
  - PII categories 492
  - profanity filter configuration 492

- prompt attacks configuration 490
- regulations 498
- review and create page 495
- sandboxing 678
- sensitive data 487
- service file 503
- step-by-step process 488
- testing actions 500–502
- test section 495, 496
- tether system 498
- versions 502

H

- Hugging Face models
  - HTTP libraries 618, 619
  - inference wrappers 619–620
  - integration via frameworks 620
  - Quarkus/LangChain4j 621
  - request-response flow 618
  - Spring AI 620
  - text generation inference (TGI) 621
- Human-in-the-loop (HITL) learning 669

I

- IDE
  - *See* Integration development environment (IDE)
- Image processing model
  - library package 416
  - meaning 416
  - OpenCV library 416
  - OpenPnP Maven repository 417
  - tesseract-platform 417, 418
- Inference optimization techniques 47
  - batching/parallelization 49
  - decoding phase 49
  - distillation 48
  - output-preserving optimizations 48

- quantization 47
- Integration development environment (IDE) 82

J

- Java applications
  - conversation memory 92–102
    - Assistant.java interface 97
    - build.gradle 96
    - ChatController 100–101
    - context 93
    - control flow 94
    - conversational systems 95
    - features 95
    - gitignore file 96
    - interactions 101–102
    - LangChainConfig 98–101
    - memory component 94, 95
    - persistent memory 96
    - requests 92
    - ServiceWithPersistentMemory 97
  - DJL
    - *See* Deep Java Library (DJL)
  - Hugging Face 618–621
  - ML
    - *See* Machine learning (ML)
  - model versions 103–105
  - ONNX Runtime (ORT) 640–643
  - prompt engineering 89–92
- Java Native Interface (JNI)
  - error handling/debugging 640
  - garbage collection 640
  - integration 638
  - native method 639
  - native shared library 638
  - primary information 638
- Java programming, development 12–13
- Java Virtual Machine (JVM) 393, 643
- Jlama

- advantages 393
- chat UI interface 400
- command-line interface 395–396
- debugging 394
- development and prototyping 394
- edge environments 394
- external services 392
- features 393
- GEMM operations 393
- lifecycle management 393
- machine learning (ML) 671
- model testing 394
- monitoring/gathering statistics 394
- monolithic architectures 394
- prerequisites 391
- quantization
  - *See* Quantization model
- Quarkus 409
  - application.properties file 410
  - boot up 410
  - curl request 412
  - package dependencies 409
  - pom.xml file 409, 410
- security 394
- simplifies distribution 394
- specialized models 392
- subcommand line 396–400
- traditional approach 392

- JNI
  - *See* Java Native Interface (JNI)
- JVM
  - *See* Java Virtual Machine (JVM)

K

- Key performance indicators (KPIs) 594
- KPIs
  - *See* Key performance indicators (KPIs)

L

- LangChain4j
  - build.gradle file 84
  - ChatController.java file 86
  - ChatRequest.java file 86
  - components 81
  - concepts/processes 81
  - Hugging Face models 621
  - initial project architecture 85–87
  - key features/capabilities 81
  - pom.xml file 83
  - prerequisites 82
  - Quarkus
    - *See* Quarkus application
  - small language model 85
  - Spring AI ecosystem 139
  - Spring Initializr dependencies 83
  - unified framework/API interface 82
- Large language model operations (LLMOps) 663
  - auditing 668
  - CI/CD pipelines 667
  - compliance, security, and auditability 665
  - core components 664
  - cultural transformation 669
  - deterministic software 667
  - DevOps/MLOps 666
  - drift and behavioral monitoring 665
  - evaluation and quality assurance 664
  - feature testing 668
  - feedback loops and active learning 665
  - human-in-the-loop (HITL) learning 669
  - lifecycle management 668
  - monitoring and observability 665
  - monitoring metrics 667
  - principles of 666
  - production/developers 666
  - prompt management/version control 664
  - tools/telemetry 668
  - unique characteristics 664

- version-control 668
- LLMOps
  - *See* Large language model operations (LLMOps)
- Large language models (LLMs) 649
  - A2A transforms 561
  - ACP model 560
  - agents 285
  - ANP model 561
  - challenges/strategies 11–12
  - concepts 15
  - context management 112
  - cultural and global sensitivities 11
  - data privacy and security 11
  - domain-specific data 38
  - ecosystem 7–10
    - benefits 10
    - frameworks 9
    - LangChain 9
    - parameter sizes 7–8
    - simple language constructs 7
    - vital aspect 9
  - ethical considerations 10
  - formats 37
  - foundational language models 4
  - fundamentals 1
  - fundamental structure 37
  - general working representation 3
  - handling outputs
    - accurate 104
    - Aircraft entity 107–108
    - automation 106
    - consistency 106
    - contextual elements 106
    - customer queries 105, 106
    - LangchainConfig.java file 108–109
    - openAiChatModel method 109
    - reliable 104
    - scalability 106
    - ServiceWithPersistentMemory 109
    - structured format 104

    - structured outputs 106, 110
- Hugging Face 5–7
- inference 47–49
- internal representation 5
- Java
    - *See* Java applications
- Jlama
    - *See* Jlama
- JNI
    - *See* Java Native Interface (JNI)
- LangChain4j
    - *See* LangChain4j
- language translation 4
- LLMOps
    - *See* Large language model operations (LLMOps)
- MCP system 560
- multimodal data
    - *See* Multimodal data
- multiple tasks 37
- observability
    - *See* Observability
- OpenAI
    - *See* OpenAI API
- patterns/relationships 3
- prompt
    - *See* Prompt engineering
- proprietary business data 111
- protocols 559, 560
- security 111–112
- self-hosting
    - *See* Self-hosting LLMs
- software development 12–13
- state-of-the-art models 38
- step-by-step process 39–41
- test
    - *See* Testing/evaluation tools
- transformer
    - *See* Transformers
- transformer architecture 4

- Transformers library 5
- Linnerud dataset
  - dataset structure 625
  - dependent variables/labels 626
  - educational purposes 627
  - independent variables 625
  - integration testing 627
  - machine learning 626
  - ONNX
    - *See* ONNX regression model
  - physiological metrics 625
- Linux programming 61–63
- LLMs
  - *See* Large language models (LLMs)
- LoRA
  - *See* Low-rank adaptation (LoRA)
- Low-rank adaptation (LoRA) 46

M

- Machine learning (ML) 2, 416, 617
  - broader transformation 670
  - cultural shift 671
  - Deep Java Library 670
  - developers 672
  - DJL
    - *See* Deep Java Library (DJL)
  - DL4J's modular architecture 671
  - evolution 669
  - hosting/inference 623–625
  - Hugging Face 5–7, 618–621
  - initiatives 672
  - Jlama 671
  - Linnerud dataset 626
  - services 543, 544
  - Vertex AI 530
- Mass Multitask Language Understanding (MMLU) 16
- MCP
  - *See* Model Context Protocol (MCP)

- Mixture-of-experts (MoE) 650
- ML
  - *See* Machine learning (ML)
- MMLU
  - *See* Mass Multitask Language Understanding (MMLU)
- Model Context Protocol (MCP)
  - capabilities 563
  - client/server layer 565
  - clients/external components 562
  - client-side function 565–567
  - communication flow 562–563
  - foundational layer 563
  - interoperability 563
  - monoliths/modular intelligence 653
  - Quarkus 579–589
    - application.properties file 581
    - browser tab 583, 584
    - connect command configuration 585, 586
    - getWeatherDetails tool 588, 589
    - implementation 579
    - Intellij IDE output 580
    - list tools button 587, 588
    - Maven package dependencies 582
    - output message 580
    - output process 582–583
    - package dependencies 579
    - prerequisites 580
    - server 582
    - tools exploration section 587
    - Weather.java 581
  - revolution 559
  - role of 563
  - server-side protocol 567–568
  - session layer 565
  - Spring AI integration
    - application file 574, 575
    - application.properties file 572, 576
    - build.gradle file 571
    - capabilities 569
    - client project 576–579

    - client/server modules 568
    - completion management 570
    - independent tooling services 572
    - prompt templates 570
    - resource management 570
    - root changes 570
    - servers 571–575
    - Spring Initializr 571, 576
    - tool context 570
    - WeatherService 573, 574
  - synchronous/asynchronous 564
  - three-tier architecture 564
  - transport layer 565
- Modular Communication Protocol
  - *See* Model Context Protocol (MCP)
- MoE
  - *See* Mixture-of-experts (MoE)
- Monoliths/modular intelligence
  - agentic patterns 652
  - architectural paradigm 654
  - open-source/cloud-hosted models 655–656
  - protocol-driven interoperability 653–654
  - RAG systems 652
  - real-world constraints 654
  - scaling 651–652
  - trends/patterns 650, 651
- Multimodal data
  - gpt-4o-audio-preview model 440–441
  - image-generation
    - attributes 433
    - FlightInfo.java 433
    - generateGreetingImage 432
    - imageModel 432
    - ImagePrompt 433
    - OpenAI image 437–439
    - scanImage method 433
    - ScanService.java file 434–437
    - user request 432
  - image processing 416–418
  - linguistic concepts 418

- process flight tickets
  - application.properties file 421
  - build.gradle file 420–421
  - ChatModel 428
  - classes/components 425–426
  - components 422
  - curl request 431
  - dependencies 419, 420
  - FlightInfo.java 423–425
  - FlightInfo object 427
  - FlightTicketController.java 430–431
  - ObjectMapper 428
  - ResponseFormat object 428
  - sample image 422
  - ScanService.java file 428–430
  - UserMessage API 427
- robotics/autonomous systems 419
- text and images 418
- transformative approach 415
- visual context 419

N

- Natural language processing (NLP) 3, 16, 38
  - capabilities 645–647
  - Maven dependenccy 646
  - named-entity recognition 646
  - OpenNLP 647
  - pretrained model 646
- NLP
  - *See* Natural language processing (NLP)

O

- Observability
  - application.properties file 596
  - characteristics 592
  - ChatClient requests
    - application.properties file 599, 604

    - build.gradle file 602–604
    - call()/stream() method 598
    - context.getName() method 605–606
    - DemoApplication.java 598, 599
    - low cardinality keys/values 601–602
    - metrics 600
    - observation section 610
    - prometheus metrics 608–610
    - properties 604
    - user/prompt contexts 606–608
  - ChatModel requests
    - high cardinality keys 613
    - implementations 611
    - low cardinality keys 612–614
    - makeChatClientRequest() method 611–612
    - metrics/traces 615
    - output process 614–615
  - CommandLineRunner pattern 596–598
  - core signals 593
  - cost management 592
  - DemoApplication.java 597
  - foundational concepts 591
  - hyperparameters 592
  - key components 595
  - logs/traces 593
  - low-*vs*. high-cardinality 594–595
  - metrics 593
  - Spring Initializr project 595, 596
  - tools/telemetry 668
  - visualizations 594
- OCR
  - *See* Optical Character Recognition (OCR)
- Ollama
  - API documentation 71–79
  - benefits 53
  - ChatService.java file 79
  - deepseek-r1 model
    - default model 65
    - information 64
    - library page 64

    - license information 66, 67
    - model-specific information 66
    - 1.5b size model 65, 67–71
  - installation process
    - documentation page 56
    - installation 59–60
    - Linux 61–63
    - source code 60–61
    - starting process 57
    - terminal/run 57, 58
    - window 59–60
  - JSON response 78
  - open-source SLM/LLM models 63
  - platforms 54–56
  - privacy 54
  - security 54
  - Spring AI ecosystem 147–152
- ONNX
  - *See* Open Neural Network Exchange (ONNX)
- ONNX-based model 623
- ONNX regression model 627
  - training/conversion process
    - load/prepare dataset 627
    - model conversion 628
    - Python 628
    - regression model 628
  - workflow process
    - approach 629
    - build.gradle file 629, 630
    - end-to-end integration 635
    - ExerciseMetricInferenceController class 634
    - ModelConfiguration.java 633
    - OnnxRuntime 633
    - PhysiologyTranslator.java 632
    - PredictionResponse.java 631
    - processOutput() method 632
    - regression inference flow 637
    - request-response control flow 636
    - request/response objects 631
    - Spring Initializr 629

- ONNX Runtime (ORT) 640
  - backend services 643
  - benefits 641
  - desktop/Android applications 643
  - key concepts 642
  - Microservices 643
  - pretrained model 641–642
- OpenAI APIs
  - compatibility 76
  - curl response 78
  - multimodal model 16
  - self-hosting LLMs 51
  - Spring AI ecosystem 147–152
  - Spring Boot
    - *See* Spring Boot application
  - transcends language models 16
  - visionaries 15–16
- Open Neural Network Exchange (ONNX)
  - ONNX Runtime (ORT) 640–643
  - regression
    - *See* ONNX regression model
- Open-source/cloud-hosted models
  - agentic systems 656
  - ecosystem 655–656
  - evaluation/benchmarking 655
  - fine-tuning and adaptation 656
  - hybrid approach 656
  - inference/serving 656
  - proprietary 655
- Optical Character Recognition (OCR) 416
- ORT
  - *See* ONNX Runtime (ORT)

P

- Parameter-efficient fine-tuning (PEFT) 46, 652
- PEFT
  - *See* Parameter-efficient fine-tuning (PEFT)
- Personally identifiable information (PII) 111

- PII
  - *See* Personally identifiable information (PII)
- Plain Old Java Objects (POJOs)
  - Spring AI ecosystem 139
- POJOs
  - *See* Plain Old Java Objects (POJOs)
- Post-training quantization 47
- Prompt engineering
  - APE tool 209–213
  - code prompting 213–223
  - contextual information 197–199
  - CoT prompting 200–202
  - definition 174
  - DemoApplication file 224–242
  - effective prompts
    - chain-of-thought (CoT) 177
    - code generation 177
    - components 175
    - contextualization 178
    - conversational flow 177
    - creation/optimization 178
    - documents 178–179
    - external context 175
    - foundational prompting techniques 176–177
    - indicator 176
    - instructions 175
    - output format 176
    - ReAct model 177
    - user input 176
    - zero-shot learning 177
  - entity mapping capabilities 192
  - few-shot/one-shot 189–191
  - Java application 92–95
  - key activities 174–175
  - model trials 661
  - practices/patterns 175
  - production applications 224
  - Quarkus 331–340
    - Comment.java 335
    - CommentResource.java 334–335

    - CommentReview.java 335
    - crafting prompts 332
    - domain types 335
    - flow diagram 332
    - output process 336–340
    - Rating.java 335
    - steps 340
    - SupportService interface 332–334
  - role prompting 193–196
  - self-consistency 202–204
  - Spring AI
    - CommandLineRunner 180
    - inner workings 179, 180
    - layers 179
    - maxTokens parameter 184–185
    - model-specific options 186–188
    - sampling controls 185
    - structured response format 185–186
    - temperature controls 181–183
    - token selection process 184
  - step-back problems 198–200
  - system prompts 191–193
  - ToT techniques 204–209
  - zero-shot 188–189
- Python 16

Q

- QAT
  - *See* Quantization-aware training (QAT)
- Quantization-aware training (QAT) 47
- Quantization model
  - definition 400
  - different options 401–402
  - Hugging Face page 402
  - Qwen2 model
    - access token page 405
    - collections 403
    - directory 408

    - files 406–407
    - list information 403
    - release link 404
    - repository home page 402
    - 0.5B model page 404, 405
- Quarkus applications 13
    - application.properties 331
    - chat memory
        - ChatMemoryProvider class 358–359
        - contextual information 362
        - conversational context 356
        - flow diagram 357
        - in-memory history 356, 358
        - POST requests 361
        - SupportService.java 360–367
    - dependencies 329
    - end-to-end process 327
    - Hugging Face models 621
    - Jlama 409–414
    - LangChain framework 328
    - log requests/responses 331
    - model context protocol (MCP) 579–589
    - openai package 329
    - pom.xml file 330, 331
    - prompt engineering 331–340
    - RAG patterns 366–388
    - tools 341–356
        - ApplicationStartup.java 346, 347
        - capabilities 341
        - CommentResource.java 346
        - Customer Entity class 342–343
        - CustomerRepository class 343
        - flow diagram 341
        - getCustomerDetail tool 350
        - initial error logs 347
        - output process 348–357
        - SupportService.java 344

R

- RAG
  - *See* Retrieval-augmented generation (RAG)
- RAM
  - *See* Random access memory (RAM)
- Random access memory (RAM) 52
- Reinforcement Learning from Human Feedback (RLHF) 46
- Retrieval-augmented generation (RAG) 9, 81, 89, 115
  - build.gradle file 133–137
  - contentRetriever method 136
  - contextual data
    - Aircraft class/flight details 116–120
    - ChatController file 121
    - ChatRequest.java file 123
    - customer query 116
    - defaultMessage 124–125
    - extractor entity class 120
    - service file 120–121
    - UML diagram 116, 117
  - embeddings 130–131
  - knowledge bases
    - application.properties file 518
    - build.gradle file 518
    - configuration file 519, 520
    - controller method 521
    - data source choices 509, 510
    - details section 509
    - implementation 507, 508
    - inputText 520
    - listing view 516
    - output process 522–524
    - overview/create page 516–517
    - retrieval/chat endpoints 509
    - retrieveAndGenerate method 521
    - service component 520
    - status views 517
    - test page 518
    - vector store 508, 509
  - LangchainConfig.java file 135
  - monoliths/modular intelligence 652
  - paradigm

    - context length 126
    - contextual length/token summary 128–130
    - cost-effective method 126
    - DeepSeek R1 models 127
    - foundational models 126
    - input prompt message 128
    - paradigm 130
    - simple query pattern 125
    - token planning/management 128
- PDF generation 507
- Print/PDF button 506
- Quarkus
    - AI service 376–388
    - contentRetriever method 374
    - embedding documents 377
    - flow diagram 366, 367
    - ingest documents 367–373
    - maxResults method 374
    - redis vector store 368
    - RetrievalAugmentor interface 373–376
    - steps 367
- Spring AI
    - airlines chat application 152
    - ChatController 170
    - ChatRequest class 169
    - ChatService file 168–169
    - chunked documents 156, 157
    - components 152
    - configuration file 156
    - DemoApplication 161–164
    - DocumentReaders 155, 156
    - embedding model 165
    - end-to-end flow 161
    - flight information 152–154
    - flow process 167
    - initializeSchema 159
    - markDocumentReader objects 159
    - matched documents 166–167
    - milvusContainer 158
    - OpenAiEmbeddingModel 159

    - response 167
    - source files and divide 154, 155
    - storage 157
    - terminal interface 171
    - vectorStore 160
  - Spring AI ecosystem 141
  - tool calling 244
  - traditional search methods 132
  - TravelX application 115
  - vector store 131–133
  - web crawling
    - data source 511–513
    - S3 data source 513–514
    - settings 512
    - strategy configuration 513, 514
    - vector store configuration 515–516
- RLHF
  - *See* Reinforcement Learning from Human Feedback (RLHF)

S

- SDKs
  - *See* Software development kits (SDKs)
- Self-hosting LLMs
  - advantages 52
  - features 52
  - GPT-based versions 52
  - Ollama
    - *See* Ollama
  - quantization 52
  - tools/frameworks 53
- Server-Sent Events (SSE) 563
- SFT
  - *See* Supervised fine-tuning (SFT)
- SLMs
  - *See* Small language models (SLMs)
- Small language models (SLMs) 4, 51
- Soft prompting 46
- Software development kits (SDKs) 16

- Spring AI ecosystem
  - boot application
    - ChatClient.Builder object 146
    - DemoApplication 146
    - dependencies 143–145
    - Initializr link 141
    - packages/libraries 141–142
    - secret key 144, 145
    - terminal running process 147
  - definition 139
  - features 140–141
  - features/utilities 139
  - Hugging Face models 620
  - integration 140
  - model context protocol (MCP) 568
  - Ollama API
    - account/payment license 147
    - application.properties file 149
    - configuration parameters 149
    - curl request 149
    - DemoApplication.java 150–152
    - dependency selection 147, 148
    - memory requirements 148
  - prompt engineering 173
  - RAG-based application 152–171
  - request-response flow 140
  - testing/evaluation tools 447–460
  - tool calling 248–250
  - vector databases 140
- Spring Boot applications
  - Chat API section
    - context message 23–25
    - curl request 22, 23
    - prompt message 25
    - reference 21–22
    - secret key creation 23
    - sidebar panel 21, 22
  - components 17
  - downloaded template 18
  - Initializr website 17, 18

- integration
  - ChatService.java 32–34
  - components 26–27
  - content 27
  - curl response 29
  - message entity 29
  - POJO objects 30–32
  - response element 27, 28
  - service layer 34–36
- IntelliJ IDE 19
- LangChain4j 83
- multimodal data 422
- prerequisites 18
- request flow 17
- retrieval-augmented generation 116
- Spring AI
  - *See* Spring AI
- startup 20
- Vertex AI platform 534
- SSE
  - *See* Server-Sent Events (SSE)
- Supervised fine-tuning (SFT) 45

T, U, V, W, X, Y, Z

- Testing/evaluation tools
  - definition 443
  - evaluation
    - modular framework 444, 445
    - performance metrics 445
    - systematic framework 444
  - libraries 446
  - probabilistic engines 443
  - Spring AI
    - chatModel and vectorStore 451
    - debugging process 451
    - EvaluationRequest 448, 452
    - evaluator interface 447
    - FactCheckingEvaluator 449, 455

    - function code 453, 456
    - implementations 448
    - Markdown document 451
    - output process 453, 455
    - prompt template 449–450
    - RelevancyEvaluator 448, 452
    - test case file 457–460
    - testFactCheckingEvaluation 454
  - strategies 447
  - summarization 446–447
- Tool calling
  - action execution 244
  - application.properties file 252
  - categories 244
  - concept of 243
  - content array 245
  - control flow 244, 245
  - declarative specification
    - ChatClient method 256
    - DateTimeTools class 253
    - description attribute 254
    - getCurrentDateTime tool 256, 258
    - method signature 254
    - MyTools file 254
    - @Tool annotation 253
  - error handling 251
  - flow of control 248–249
  - implementation details 282
  - information retrieval 244
  - JSON strings 246
  - models 246–247
  - Open AI dependency 252
  - OpenWeather service
    - account sign-up 259
    - API data 261
    - application.properties file 260
    - call current weather data 262
    - Clouds.java 274
    - Coord.java 268
    - DemoApplication.java 280–282

    - input parameters 263
    - key creation 261
    - Main.java 275
    - MyTools.java file 278, 279
    - output process 281
    - Rain.java 274
    - restTemplate class 277
    - source code 263–266
    - Sys.java 272
    - verification email 259, 260
    - WeatherData 278
    - WeatherData.java 269
    - Weather.java 267
    - weather response data 277
    - Wind.java file 266–268
  - providers 245
  - Quarkus application 341–356
  - Spring AI 248–250
  - structured feedback 251–252
  - travel activities and goals 258
  - try-catch blocks 251
- ToT
  - *See* Tree of thoughts (ToT)
- Transformers
  - architecture 42
  - components 41
  - decoder process 43
  - encoder process 42
  - evolution 43–45
  - fine-tuning 45–47
  - identical layers 43
  - Meta LLaMA series 44
  - policy optimization 44
  - sentences/stories 41
  - target language 43
- Tree of thoughts (ToT) 204–209

Previous chapter

< 20. Architectures of Tomorrow: From Monoliths to Modular Minds